COMP3821/9801

Workshop Week 9

Convolutions and Flow Networks

Copyright Disclaimer. *Reproducing, publishing, posting, distributing, translating this document is an infringement of copyright and will be referred to UNSW Student Conduct and Integrity for action.*

Guided Problems

The demonstrators will solve the following problems. For the rest of the workshop, work in small groups to solve the remaining workshop exercises. There are optional problems at the end of this document for further practice.

Guided Problem

It is the Year of the Snake, and you just so happen to have a collection of n coloured snake figurines. Each figurine is coloured one of c colours. Your friend wants to keep a set of k snake figurines from your collection. Two sets of snake figurines are considered the same if they contain the same number of figurines of each colour.

Describe and analyse a polynomial-time algorithm (in terms of c, k, n) that returns the number of different ways of choosing k snake figurines. Your input is an array $\textsc{Snake}[1..c]$ and an integer k, where $\textsc{Snake}[i]$ denotes the number of snake figurines of colour i. Your output is a non-negative integer.

Note. *You cannot assume that c or k are constants. You may assume that $k \leq n$.*

Guided Problem

Let $G = (V, E)$ be a *directed* and acyclic graph, where $|V| = n$ and $|E| = m$. Let $\mathscr{P} = \{P_1, \ldots, P_k\}$ be a collection of disjoint paths in G. We say that $\mathscr{P}$ is a *path covering* if every vertex $v \in V$ in G is covered in *exactly* one path P_i. In other words, a path covering is a collection of disjoint paths that cover all vertices in G. Describe and analyse a polynomial-time algorithm that returns the minimum number of paths such that each vertex in V lies in *exactly* one path P_i.

Workshop Exercises

1. Let A be a binary array of n elements; that is each element of A is either 0 or 1. A triplet of distinct indices $1 \leq i, j, k \leq n$ is called a *nice* triplet of A if $A[i] = A[j] = A[k] = 1$ and $k = i + j$. Describe and analyse a polynomial time algorithm that decides if a nice triplet of A exists.

2. For two binary strings (strings containing only 0 and 1) A and B of length n, the *Hamming distance* of A and B is the number of indices where A and B disagree. For example, if $A = \texttt{1100}$ and $B = \texttt{0110}$, then the Hamming distance of A and B is 2 since A and B disagree at indices 1 and 3.

$$\begin{array}{rcccc} A: & 1 & 1 & 0 & 0 \\ B: & 0 & 1 & 1 & 0 \end{array}$$

   We have two binary strings: the first is the *text* string T, which has length n, and the second is the *pattern* string P, which has length m. The pattern string is no larger than the text string, so $m \leq n$. We want to search for a substring of the text which is closest to the pattern by Hamming distance.

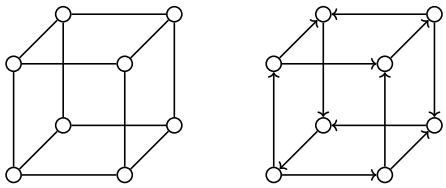
   For example, if we have $T = \texttt{001100010}$ and $P = \texttt{11010}$, then the closest match to P in T is $T[3] \ldots T[7]$, which has a Hamming distance of 1:

T: 0 0 1 1 0 0 0 1 0
P: 1 1 0 1 0

Describe and analyse a polynomial-time algorithm to return the minimum Hamming distance between the pattern P and a substring of T of length m.

3. Let $G = (V, E)$ be an undirected graph. A *k-orientation* assigns each edge a direction such that every vertex of G has *at most* k incoming edges. Describe and analyse a polynomial-time algorithm to find the smallest k for which G has a k-orientation.

   For example, the cube graph should return $k = 2$, as shown below.



   Hint. *Fix a value for k first and solve the decision problem of deciding whether a k-orientation exists.*

4. Let $G = (V_1 \cup V_2, E)$ be a bipartite graph, where $|V_1| = |V_2| = n$ and $|E| = m$. Let the vertices of V_1 be denoted by $v_1, \ldots, v_n$ and let the vertices of V_2 be denoted by $w_1, \ldots, w_n$. A matching M is *planar* if every pair of edges in M do not cross. More formally, all pair of edges (v_i, w_j) and $(v_{i'}, w_{j'})$ in M satisfy the requirement that $i < i'$ if and only if $j < j'$.

   Describe and analyse a polynomial-time algorithm that returns the minimum number of planar matchings $M_1, \ldots, M_k$ such that each edge in E lies in *exactly* one matching M_i.

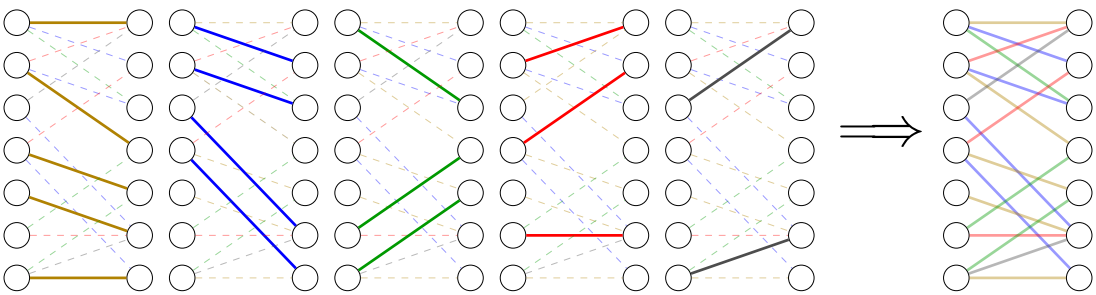


Figure 1: *A collection of five planar matchings.*

There are at least four approaches.

Approach 1. *Reduce to a problem that you have previously seen; this should have running time $O(m^3)$.*

Approach 2. *Use Dilworth's (or Mirsky's) theorem. Construct an appropriate poset, and use the chain-antichain duality. The minimum number of* ***chains*** *that cover the poset should correspond to the maximum length* ***antichain****. This should have running time $O(m^2)$.*

Approach 3. *Use dynamic programming and reduce to the longest decreasing subsequence problem. This should have running time $O(m \log m)$.*

Approach 4. *Use ordered dictionaries or van Emde Boas trees for the competitive programming enthusiasts. This should have running time $O(m \log \log n)$.*

5. Let $G = (V, E)$ be a *directed* graph with n vertices and m edges. Let $\mathscr{C} = \{C_1, \ldots, C_k\}$ be a collection of disjoint cycles in G. We say that $\mathscr{C}$ is a *cycle covering* if every vertex $v \in V$ in G is covered in *exactly* one cycle C_i. In other words, a cycle covering is a collection of disjoint cycles that cover all vertices in G.

   (a) Let $E' \subseteq E$ be a set of edges in G. Prove that E' forms a cycle covering if and only if every vertex has exactly one incoming edge and one outgoing edge in E'.

   (b) Describe and analyse a polynomial-time algorithm to decide whether G contains a cycle covering. You are not asked to return the covering (if one exists), just whether G contains one.

   Hint. *Use bipartite matching to check for the existence of a set of edges that satisfy the condition from part (a).*

Optional Exercises

These exercises will not contain solutions. We encourage that you post on the forums to discuss the approaches for these problems.

6. You have a set of n coins in a bag, each coin having a value between 1 and m, where $m \geq n$. Some coins might have the same value. You pick two coins at random, with replacement, and record the sum of their values. In other words, once you record the value of the first coin, you immediately place it back into the bag and pick up another coin.

   Describe and analyse a polynomial-time algorithm that returns the set of possible sums that can be achieved.

   Note. *You know the set of coin values in the bag ahead of time.*

7. Let $A[1..n]$ be an array of boolean values. An *inversion* in A is a pair of indices (i, j) such that $i < j$, but $A[i] = \textsc{True}$ and $A[j] = \textsc{False}$. The *length* of an inversion (i, j) is given by $j - i$. For each length $1 \leq \ell \leq n - 1$, your task is to compute the number of inversions of length ℓ.

   Describe and analyse an $O(n \log n)$ algorithm that returns an array B such that $B[\ell]$ is the number of inversions of length ℓ.

8. Let A and B be two sets of integers. Define the *Minkowski sum* $A + B$ as the set of all pairwise sums of integers in A and B; that is, we define

$$A + B = \{a + b \mid a \in A,\ b \in B\}.$$

   (a) Without further restrictions to the input, describe an $O(n^2 \log n)$ algorithm that returns the number of elements in $A + B$, where $|A| + |B| = n$.

   (b) The previous part assumed that the integers in A and B were *unbounded*. For each $x \in A \cup B$, assume now that we know that $|x| \leq M$; that is, the integers in $A \cup B$ are bounded above by M and below by $-M$. Describe an $O(M \log M)$ algorithm that returns the number of elements in $A + B$.

   Note. *The $O(n^2 \log n)$ algorithm is currently the fastest known algorithm for the Minkowski sum problem. We often study the Minkowski sum in additive combinatorics.*

9. Let $A[1..n]$ be an array of n positive integers. An *interval sum* of A is a contiguous subarray $A[i..j]$ (where $1 \leq i < j \leq n$) of A. An *interval sum* is the sum of the integers of the interval. Suppose that we have computed all $\Theta(n^2)$ interval sums, but some of the interval sums might coincide. We are tasked with the problem of counting the number of *distinct* interval sums.

   (a) Describe and analyse an $O(n^2 \log n)$ algorithm to compute the number of *distinct* interval sums of A.

   Hint. *Prefix sum.*

   (b) Now, suppose that $M = \sum_{i=1}^{n} A[i]$. Describe and analyse an $O(M \log M)$ algorithm to compute the number of *distinct* interval sums of A.

   Hint. *Modify the prefix sum and use convolutions.*

10. Assume that the characters come from alphabet Σ. Let $T[1..n]$ be an array of n characters, and let $P[1..m]$ be an array of m characters where some of the characters in P and T may contain a special wildcard $(?)$ character. Assume that

$m \leq n$. The task is to find all substrings of T where the substring P occurs; note that a wildcard symbol matches *any* character. For example, consider the following strings.

- **Text.** `ILOVETHEFASTFOURIERTRANSFORM`
- **Pattern.** `A??FO`

There are two positions where the pattern can be matched, highlighted below.

`ILOVETHEFASTFOURIERTRANSFORM`

For our purposes, we may assume that the characters come from an alphabet of positive integers; that is, the characters are positive integers which are separated by a delimiter.[1]

(a) Describe and analyse a polynomial-time algorithm that finds all positions where P can occur inside T.

(b) Now, assume that both P and T are allowed to have wildcard symbols. Describe and analyse a polynomial-time algorithm to find all positions where P can occur inside T.

11. Let $F = (V, E)$ be an (s, t)-flow network. For each directed edge $u \to v \in E$, let $v \to u$ denote the edge in the opposite direction with $c(v \to u) = c(u \to v)$. We call such an edge an *anti-parallel* edge. For a flow network (possibly with anti-parallel edges), show that there exists an equivalent flow network without anti-parallel edges.

12. Let $F = (V, E)$ be an (s, t)-flow network, and let $f : E \to \mathbb{Z}^+ \cup \{0\}$ be a flow in F. You may assume that F has no anti-parallel edges; otherwise, we can construct a flow network as above.

    (a) Describe and analyse a linear-time algorithm to determine if f is a *maximum flow*.

    (b) Describe and analyse a linear-time algorithm to determine if f is the *unique* maximum flow.

13. Optisigi and his wife Jane McStreet have decided to found a university after their son was rejected from the University of New South Wales. Since their son really wanted to do software engineering, they decided to name it *The University of Nimble Software Engineers* (UNSWE).

    It's a new year at UNSWE! Due to their increasing reputation in teaching quality, n students have enrolled to begin their studies. To maintain the reputation and ensure that each student gets the utmost care and attention, UNSWE requires that every student live in one of the m dorm rooms on campus.

    The dorm rooms are spread uniformly around the campus, and each dorm room can accommodate at most k students. Each of the m dorm rooms have a different theme, and each student has different thematic preferences. To account for this, UNSWE allows each student to give a list of dorms that they are happy to stay in. Naturally, UNSWE wants to maximise the number of happy students by allocating them to one of their desired dorm rooms.

[1] This is more of a technical issue than anything else; we may think of this as a way of hashing the characters in the alphabet. We don't need to care about the construction of the hash function.

More formally, you are given an $n \times m$ array of booleans Preferred, where Preferred$[i][j]$ is true if student i is happy to stay in dorm j, or false otherwise. You are also given an array of m non-negative integers Capacity, where Capacity$[j]$ denotes the maximum number of students that can live in dorm j.

Describe and analyse a polynomial-time algorithm that finds the maximum number of happy students.

14. Googatlanvasoft has hired n interns to work over the summer, and there are currently m projects that each of the interns could work on. Each intern can be assigned to any number of the projects. It is valid to assign an intern to no projects. It is valid to assign an intern to all m projects. Each project requires a specific number of interns.

    At the start of the summer, each intern was asked to choose a selection of projects that they were interested in working on.

    More formally, you are given an $n \times n$ boolean array Interest, where Interest$[i, j]$ is True if intern i is interested in project j or False otherwise. You are also given an array of m positive integers Required, where Required$[j]$ is the number of interns required for project j. Each entry of this array is at most n.

    Describe and analyse a polynomial-time algorithm which determines whether or not there is a satisfactory allocation of interns to projects. An allocation is satisfactory if interns are only assigned to projects that they were interested in working on, and each project has exactly the required number of interns.

15. Let M be an $n \times n$ matrix such that each entry of M is either 0 or 1. Let M_{ij} denote the entry in the ith row and the jth column of M. We define the following actions on M:

    - RowSwap(i, j): Swaps the entries of M_{ik} with M_{jk} (and vice versa) for each $k = 1, \ldots, n$.
    - ColSwap(i, j): Swaps the entries of M_{ki} with M_{kj} (and vice versa) for each $k = 1, \ldots, n$.

    A matrix is *swappable* if there exist a sequence of RowSwap and ColSwap operations such that the resulting matrix has the element 1 along the diagonal. Given a matrix M, describe and analyse a polynomial-time algorithm that decides whether M is swappable or not.

    Note. *You are not asked to return the sequence of operations (if it exists), just whether one such sequence exists.*

16. Suppose that you are given an $n \times n$ chessboard $A[1..n, 1..n]$, where a subset of squares have been removed. You need to describe an algorithm to place as many pieces of some type as possible such that no two pieces attack each other. A piece can be placed on square (i, j) if and only if $A[i, j] = \text{True}$.

    (a) Describe and analyse a polynomial-time algorithm to place as many *rooks* as possible. Recall that a rook on square (i, j) can attack every square in the same row or column; that is, the rook can attack squares (i, k) and (k, j) for all $1 \leq k \leq n$.

    (b) Describe and analyse a polynomial-time algorithm to place as many *bishops* as possible. Recall that a bishop on square (i, j) can attack every square on

the same diagonals; that is, the bishop can attack squares $(i+k, j+k)$ and $(i+k, j-k)$ for all $1 \leq k \leq n$.

(c) Describe and analyse a polynomial-time algorithm to place as many *knights* as possible. Recall that a knight on square (i, j) can attack squares $(i \pm 1, j \pm 2)$ and $(i \pm 2, j \pm 2)$.

17. Let X be a set of n integers. You want to store these integers into an array A of size n.

    You can only do so by calling one of three oracles h_1, h_2, h_3, each of which takes an element from X and returns an integer from 1 to n in constant time.

    We say that this collection of oracles is *perfect* if

    - Each element $x \in X$ is stored at **exactly one** of $A[h_1(x)], A[h_2(x)], A[h_3(x)]$.
    - Each $A[i]$ stores **exactly one** element of X.

    Given the set X and the set of three oracles, design and analyse an $O(n^2)$ algorithm to decide if the collection of oracles is perfect.

18. Let $G = (V, E)$ be an *undirected* graph, and let $u, v, w \in V$ be three distinct vertices. Describe an $O(m)$ algorithm to determine whether there exist a simple path from u to v passing through w.

    Hint. *Reduce this to a problem you have previously seen, maybe in lectures...*

    Note. *The fact that G is **undirected** makes the problem easy. The problem becomes* NP*-hard if G was directed.*

19. Let $F = (V, E)$ be an (s, t)-flow network. In this problem, our task is to be able to classify vertices in V by a special kind of property that we will define.

    - A vertex v is *saucy* if v appears on the source side of *every* minimum cut; that is, if (S, T) is a minimum cut of F, then $v \in S$.
    - A vertex v is *sinky* if v appears on the sink side of *every* minimum cut; that is, if (S, T) is a minimum cut of F, then $v \in T$.
    - A vertex v is *saunky* if v is neither saucy nor sinky. In other words, there exist at least one minimum cut (S, T) for which $v \in T$ and there exist at least one minimum cut (S', T') for which $v \in S'$.

    Describe and analyse a polynomial-time algorithm to classify each vertex as either saucy, sinky, or saunky.

    Note. *There can exist exponentially many minimum cuts in a flow network, so do not try and enumerate over all possible minimum cuts.*

20. Let $G = (V, E)$ be a bipartite graph. Describe and analyse a polynomial-time algorithm that computes the size of the maximum independent set of G.

21. In this exercise, we prove a fundamental result in matching and graph theory. Let $G = (V, E)$ be an undirected graph with no vertex of degree zero. We define the following notations:

    - $\alpha(G)$: the size of the largest *independent set* in G; that is, $\alpha(G)$ is the maximum set of vertices $S \subseteq V$ such that no two vertices in S are connected by a direct edge.
    - $\alpha'(G)$: the size of the largest *matching* in G; that is, $\alpha'(G)$ is the maximum set of edges $T \subseteq E$ such that no two edges in T share a common endpoint.

- $\beta(G)$: the size of the smallest *vertex cover* in G; that is, $\beta(G)$ is the minimum set of vertices $S \subseteq V$ such that every edge is incident to at least one vertex in S.
- $\beta'(G)$: the size of the smallest *edge cover* in G; that is, $\beta'(G)$ is the minimum set of edges $T \subseteq E$ such that every vertex is incident to at least one edge in T.

(a) Prove that $\alpha(G) + \beta(G) = |V|$.

Hint. *Prove that X is an independent set if and only if $V \setminus X$ is a vertex cover.*

(b) Prove Gallai's theorem: *if $G = (V, E)$ is an undirected graph with no vertex of degree zero, then $\alpha'(G) + \beta'(G) = |V|$.*

Hint. *Let M be a maximum matching in G; use this to construct an edge cover L to give you one inequality. Then let L be an edge cover of G; use this to construct a matching M to give you the other inequality.*

(c) Let $G = (V, E)$ be a *bipartite* graph with no vertex of degree zero. Use Gallai's theorem to conclude that $\alpha(G) = \beta'(G)$.

(d) Hence, describe and analyse a polynomial-time algorithm to find the size of the smallest edge cover of a bipartite graph G, or report that no such edge cover exists.

22. Let $G = (V, E)$ be a directed and acyclic graph with a single source vertex s. A *spanning binary tree* is a spanning tree T of G such that every vertex in G has at most two outgoing edges in T and every vertex of G *except* s has exactly one incoming edge in T. Describe and analyse a polynomial-time algorithm to decide if G contains a spanning binary tree.

23. Let $A[1..m][1..n]$ be a two-dimensional array of non-negative real numbers. We want to *round* A so that every element of A is an integer without changing the sum of entries in any row or column. In other words, we want to replace x with either $\lfloor x \rfloor$ or $\lceil x \rceil$.

$$\begin{bmatrix} 1.2 & 3.4 & 2.4 \\ 3.9 & 4.0 & 2.1 \\ 7.9 & 1.6 & 0.5 \end{bmatrix} \implies \begin{bmatrix} 1 & 4 & 2 \\ 4 & 4 & 2 \\ 8 & 1 & 1 \end{bmatrix}$$

Given a real-valued array A, describe and analyse a polynomial-time algorithm to compute a rounding of A as defined above, or report that no such rounding is possible.

24. Let F be a flow network with integer capacities. Your task is to return the minimum cut with the smallest number of edges that cross the cut.

(a) Consider the new flow network $\hat{F}$, defined by the new capacity function

$$\hat{c}(x \to y) = c(x \to y) \cdot (m + 1) + 1,$$

where m denotes the number of edges in F. Show that if (S, T) is *not* a minimum cut of F, then (S, T) cannot be a minimum cut of $\hat{F}$.

This shows that any minimum cut of $\hat{F}$ must be a minimum cut of F.

(b) Show that the minimum cut of F with the minimum number of edges will have the lowest cut capacity in $\hat{F}$.

Hence, the minimum cut of $\hat{F}$ will be the minimum cut with the smallest number of edges that cross the cut.

25. Let F be a flow network with integer edge capacities. An edge is *expanding* if increasing its capacity by 1 increases the maximum flow in F by 1. An edge is *reducing* if decreasing its capacity by 1 decreases the maximum flow by 1.

    (a) Does every flow network F contain at least one expanding edge? Prove your answer, or provide a counterexample.

    (b) Does every flow network F contain at least one reducing edge? Prove your answer, or provide a counterexample.

    (c) Given a flow network F and a *maximum flow* f, describe and analyse an $O(m)$ algorithm to find all expanding edges.

    (d) Given a flow network F and a *maximum flow* f, describe and analyse an $O(mn)$ algorithm to find all reducing edges.

26. Let $F = (V, E)$ be an (s, t)-flow network. A minimum cut (X, Y) is *minimal* if there is no minimum cut (X', Y') such that $X' \subset X$ and $Y' \subset Y$. In other words, a minimal minimum cut is a minimum cut where there is no minimum cut that is a proper subset.

    (a) Prove that a minimal minimum cut is *unique*.

    Hint. *Prove that, if S and S' are minimum cuts, then $S \cap S'$ is also a minimum cut.*

    (b) Describe and analyse a polynomial-time algorithm to compute the unique minimal minimum cut.

27. You are given a collection of jobs J, indexed from 1 to n, and you have two processors available to run them on. You can run job i on the first processor at a cost of $A[i]$, or you can run job i on the second processor at a cost of $B[i]$.

    In addition, if two jobs i and j are assigned on different processors, then they incur a *network cost* of $C[i, j]$. Note that $C[i, j] = C[j, i]$. Therefore, our goal is to assign *some* subset $S \subseteq J$ of jobs onto processor 1 and the remaining $J \setminus S$ jobs onto processor 2 such that the total cost

    $$\sum_{i \in S} A[i] + \sum_{i \notin S} B[i] + \sum_{i \in S} \sum_{j \notin S} C[i, j]$$

    is minimised.

    Describe and analyse a polynomial-time algorithm to assign the jobs onto the two processors such that the total cost is minimised.

28. You are given a set C of n courses, and each course $i \in C$ has an associated benefit $B[i]$, which can be positive or negative. In other words, some courses may be poorly run that it incurs negative benefit to choose it. Certainly, different courses have different prerequisites. Therefore, in order to choose course i, you must also choose *all* of its prerequisite courses as well. A course may have many prerequisites, and a course may be a prerequisite to many courses.

    A set $X \subseteq C$ of courses is *feasible* if the prerequisite of every course in X also belongs in X. Finally, the total profit of X is the sum of the individual benefits

of courses in X; that is, the total profit of X is given by

$$\textsc{Profit}(X) = \sum_{i \in X} B[i].$$

Note. You are given the prerequisite list as a two-dimensional boolean array $\textsc{Prereq}[i][j]$, which returns $\textsc{True}$ if and only if i is a prerequisite of j, and $\textsc{False}$ otherwise. You are also given a one-dimensional integer array $B[i]$, which returns the benefit of course i.

Describe and analyse a polynomial-time algorithm that returns a feasible subset of courses such that the total profit is maximised.

29. Let $G = (V, E)$ be an undirected graph. For a subset $S \subseteq V$ of vertices, let $e(S)$ denote the number of edges $(u, v) \in E$ such that $u, v \in S$. Define the quantity

$$\mathsf{coh}(S) := \frac{e(S)}{|S|}.$$

(a) Given a rational number α, describe and analyse a polynomial-time algorithm to decide if there exist a set S such that

$$\mathsf{coh}(S) \geq \alpha.$$

Note. *Yes, we can still solve flow problems on flow networks with rational capacities.*

(b) Hence, describe and analyse a polynomial-time algorithm to compute

$$\max_{S \subseteq V} \{\mathsf{coh}(S)\}.$$

30. Let T be a set of n teams in a sports league. A team i is *lost* if i cannot possibly come first or tied for first, where a team comes first if no other team has more wins. For each team $i \in T$, let w_i denote the number of wins and for each pair of teams $i, j \in T$, let g_{ij} denote the number of games that i and j have left to play. Given a team k, describe and analyse a polynomial-time algorithm that determines if k is lost.